

Reviewer Pack — Finite-Field Rank Threshold for ACC^0 Circuit Complexity

Entry ac07c338962c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-11 10:35:06 UTC

Finite-Field Rank Threshold for ACC^0 Circuit Complexity

Entry ID: ac07c338962c **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-11 10:35:06 UTC

1. Conjecture

Field A (mathematical branch): Finite Field Geometry **Field B** (complexity object): ACC^0 Circuit Size for Sipser Function

Statement:

For the Sipser function on n variables, the rank of its truth-table matrix over $\text{GF}(2)$ is $\Omega(2^{\{n/2\}})$, and this rank is exponentially larger than the minimal ACC^0 circuit size computing it.

Rationale (proposer’s reasoning):

The high rank of the truth-table matrix over $\text{GF}(2)$ reflects the function’s algebraic structure, which may inherently resist efficient ACC^0 computation due to the limited depth of such circuits.

Taxonomy category: META_COMPLEXITY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 6507095556e19b53

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math

def gaussian_elimination(matrix):
    n = len(matrix)
    for i in range(n):
        if matrix[i][i] == 0:
            return -1 # Singular matrix, cannot compute rank
        for j in range(i + 1, n):
            factor = matrix[j][i] / matrix[i][i]
            for k in range(i, n):
                matrix[j][k] -= factor * matrix[i][k]
    rank = sum(1 for row in matrix if any(row))
    return rank

def sipser_function(n, x):
    result = 0
    for i in range(n):
        result ^= (x >> i) & 1
    return result ^ 1

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    total_rank = 0
    instances_tested = 0
    conjecture_holds = True
    counterexample = ""

    for n in n_values:
        rank = 0
        for _ in range(30):
            x = random.getrandbits(n)
            truth_table = [[sipser_function(n, i) ^ sipser_function(n, j) for j in range(n)] for i in range(n)]
            rank = gaussian_elimination(truth_table)
            if rank == -1:
                conjecture_holds = False
                counterexample = "singular_matrix"
                break
            total_rank += rank
        instances_tested += 30 * n

    mean_rank = total_rank / instances_tested
    support_fraction = sum(1 for n in n_values if rank >= 2**(n/2)) / len(n_values)
```

```

    return {
        "metric_name": "rank",
        "metric_value": mean_rank,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]]
    if not seeds:
        primes = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73,
        seeds = primes[:30]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, **{result}}}")
        results.append(result)

    mean_rank = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_rank} std=0.0 support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{r['counterexample']}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out without producing results; insufficient data to evaluate support fraction or detect counterexamples. | next: Increase timeout duration and re-run experiment with deterministic seed configuration

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	111429
2	propose	ollama_remote	qwen3:8b	0	0	96643
3	preregistration	ollama_remote	qwen3:8b	0	0	24134
4	novelty	ollama_remote	qwen3:8b	0	0	20919
5	novelty	ollama_remote	qwen3:8b	0	0	16467
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10762
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9374
8	judge	ollama_remote	qwen3:8b	0	0	17151

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 306879 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/ac07c338962c.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/ac07c338962c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/ac07c338962c.tar.gz` (if generated)